

Advanced Database Systems - MIT 8107

Redis Key-Value Store Lab Guide

Duration: 30-60 minutes

Data Model: Key-Value Store

Table of Contents

1. Introduction
2. Setup Instructions
3. Basic Operations
4. Applied Scenario: Real-Time Session Management System
5. Advanced Operations
6. Group Contribution Summary

1. Introduction

What is a Key-Value Database?

A key-value database is the simplest type of NoSQL database that stores data as a collection of key-value pairs. Each key is unique and acts as an identifier to retrieve its associated value. Think of it like a dictionary or hash map.

Key Characteristics:

- ☐ Simple structure: Key → Value
- ☐ Fast lookups: $O(1)$ time complexity
- ☐ Flexible values: Can store strings, numbers, lists, sets, hashes, etc.
- ☐ High performance: Optimized for speed and scalability

Why Redis?

Redis (Remote Dictionary Server) is an open-source, in-memory key-value store known for: extremely fast read/write operations (sub-millisecond latency), rich data types (strings, hashes, lists, sets, sorted sets), built-in persistence options, wide industry adoption (Twitter, GitHub, Stack Overflow)

When to Use Key-Value Stores?

Key-value DBs are ideal for session management and caching, real-time analytics and leaderboards, rate limiting and API throttling, shopping carts, User preferences and profiles and Message queues.

On the other hand, they are not a best suite for complex queries with multiple conditions, relationships between entities, data requiring frequent aggregations.

2. Setup Instructions:

Prerequisites

1. VS Code: [link](#)
2. Git and Git Bash: [link](#)
3. GitHub Desktop: [link](#)
4. WSL (for Windows OS): [link](#)
5. Docker: [link](#)
6. VS Code extensions:
7. Docker and YAML

Step 1: Pull Redis Docker Image

Enter the following command to pull the redis docker image

`docker pull redis:7.2-alpine`

Expected Output

```
(base) dieudonne@dieudonne-HP-Pavilion-Laptop-15-cs3xxx:~/MIT8107-Lab-on-NoSQL-kifaru-byters$ docker pull redis:7.2-alpine
7.2-alpine: Pulling from library/redis
4f3ec5172bc6: Pull complete
6193948d8b6e: Pull complete
19502926555b: Pull complete
f637881d1138: Pull complete
8e886e49e824: Pull complete
1332afce1652: Pull complete
42fb427016d3: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:8939c1e45ebd9585e6888d47e54c41c16ace6d159dd5038d5770b7531e797c0e
Status: Downloaded newer image for redis:7.2-alpine
docker.io/library/redis:7.2-alpine
(base) dieudonne@dieudonne-HP-Pavilion-Laptop-15-cs3xxx:~/MIT8107-Lab-on-NoSQL-kifaru-byters$
```

☐	Name	Tag	Image ID	Created	Size	Actions
☐	 redis	7.2	5a12cb25b33b	11 days ago	171.97 MB	  
☐	 customized-apache-http	1.0	7a339bdcc746	22 days ago	707.84 MB	  
☐	 postgres	17.2	3267c505060a	10 months ago	614.42 MB	  
☐	 customized-mysql	1.0	90501fc5e693	1 year ago	811.43 MB	  
☐	 redis	7.2-alpine	8939c1e45ebd	25 days ago	59.76 MB	  

Step 2: Run Redis Container

Enter the following command to start Redis container with persistent storage:

```
docker run -d \  
--name redis-lab \  
-p 6379:6379 \  
-v redis-data:/data \  
redis:7.2-alpine redis-server --appendonly yes
```

Command Breakdown:

- `-d`: Run in detached mode (background)
- `--name redis-lab`: Name the container for easy reference
- `-p 6379:6379`: Map port 6379 (host:container)
- `-v redis-data:/data`: Create volume for data persistence
- `--appendonly yes`: Enable AOF (Append Only File) persistence

Expected Output

```
(base) dieudonne@dieudonne-HP-Pavilion-Laptop-15-cs3xxx:~  
/MIT8107-Lab-on-NoSQL-kifaru-byters$ docker run -d --na  
me redis-lab -p 6379:6379 -v redis-data:/data redis  
:7.2-alpine redis-server --appendonly yes  
6ed04e6a68e244634088fffdadb07f53ecb46883c974e0ea536a64316  
0a3f809  
(base) dieudonne@dieudonne-HP-Pavilion-Laptop-15-cs3xxx:~
```

Step 3: Verify Installation

Enter the following command to check if container is running

```
docker ps
```

Expected Output:

```
(base) dieudonne@dieudonne-HP-Pavilion-Laptop-15-cs3xxx:~/MIT8107-Lab-on-NoSQL-kifaru-byters$ docke  
● r ps  
CONTAINER ID   IMAGE                                COMMAND                                CREATED        STAT  
US            PORTS  
6ed04e6a68e2   redis:7.2-alpine                    "docker-entrypoint.s..." 3 minutes ago  Up 3  
minutes      0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp  redis-lab  
f30549476852   redis:7.2-alpine                    "docker-entrypoint.s..." 5 minutes ago  Up 5  
minutes      6379/tcp  musing_blackburn  
eeddff816f67   postgres:17.2                       "docker-entrypoint.s..." 2 weeks ago   Up 3  
2 minutes    0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp  postgres-17.2  
a7ce1ab2f182   customized-apache-httpd-plus-php:1.0 "docker-php-entrypoi..." 2 weeks ago   Up 3  
2 minutes    0.0.0.0:8002->80/tcp, [::]:8002->80/tcp      apache-2.4-php-8.2  
8308e66f41db   customized-mysql:1.0                "docker-entrypoint.s..." 2 weeks ago   Up 3  
2 minutes    0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp  mysql-8.0.40-debian  
○ (base) dieudonne@dieudonne-HP-Pavilion-Laptop-15-cs3xxx:~/MIT8107-Lab-on-NoSQL-kifaru-byters$
```

Step 4: Connect to Redis CLI

Enter the following command to connect to Redis command-line interface

```
docker exec -it redis-lab redis-cli
```

You should see the Redis prompt:

```
127.0.0.1:6379>
```

Step 5: Test Connection

Inside Redis CLI, test with PING command

```
127.0.0.1:6379> PING
```

Expected Output:

```
NEHEMA@DESKTOP-P67G70S MINGW64 ~/MIT8107-Lab-on-NoSQL-kifaru-byters (main)
$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> PING
PONG
```

3. Basic Operations (CRUD)

Understanding Redis Data Types

Before we dive into CRUD operations, let's understand Redis's core data types:

1. String - Simple key-value pairs
2. Hash - Key with multiple field-value pairs (like objects)
3. List - Ordered collection of strings
4. Set - Unordered collection of unique strings
5. Sorted Set - Set ordered by score

CREATE Operations

A. String Operations

Enter the following command to set a simple key-value pair

```
SET user:1001:name "Alice Johnson"
```

Enter the following command to set with expiration (EX = seconds, PX = milliseconds)

```
SET session:abc123 "user_data" EX 3600
```

Enter the following command to set only if key doesn't exist (NX = Not eXists)

```
SET user:1001:created_at "2025-11-28" NX
```

Enter the following command to set multiple key-value pairs at once

*MSET user:1001:email "alice@email.com" user:1001:age "28"
user:1001:country "Kenya"*

Expected Output:

```
127.0.0.1:6379> SET user:1001:name "Alice Johnson"
OK
127.0.0.1:6379> SET session:abc123 "user_data" EX 3600
OK
127.0.0.1:6379> SET user:1001:created_at "2025-11-28" NX
OK
127.0.0.1:6379> MSET user:1001:email "alice@email.com" user:1001:age "28" user:1001:country "Kenya"
OK
```

B. Hash Operations (Storing Objects)

Enter the following command to create a hash with multiple fields

*HSET user:1002 name "Bob Mwangi" email "bob@email.com" age 32
country "Kenya"*

Enter the following command to add individual fields to hash

*HSET user:1002 role "developer"
HSET user:1002 department "Engineering"*

Expected Output:

```
127.0.0.1:6379> HSET user:1002 name "Bob Mwangi" email "bob@email.com" age 32 country "Kenya"
(integer) 4
127.0.0.1:6379> HSET user:1002 role "developer"
Invalid argument(s)
127.0.0.1:6379> HSET user:1002 role "developer"
(integer) 1
127.0.0.1:6379> HSET user:1002 department "Engineering"
(integer) 1
```

C. List Operations

Enter the following commands to create a list (push from left/right)

*RPUSH user:1001:login_history "2025-11-28 09:00:00"
RPUSH user:1001:login_history "2025-11-28 14:30:00"
LPUSH user:1001:notifications "Welcome to the platform!"*

Expected Output:

```
NEHEMA@DESKTOP-P67G70S MINGW64 ~/MIT8107-Lab-on-NoSQL-kifaru-byters (main)
$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> RPUSH user:1001:login_history "2025-11-28 14:30:00"
(integer) 2
127.0.0.1:6379> LPUSH user:1001:notifications "Welcome to the platform!"
(integer) 2
```

D. Set Operations

Enter the following command to add members to a set

SADD user:1001:skills "Python" "Java" "Docker" "Redis"

SADD user:1002:skills "JavaScript" "Python" "React"

Expected Output:

```
(integer) 4
127.0.0.1:6379> SADD user:1001:skills "Python" "Java" "Docker" "Redis"
(integer) 4
127.0.0.1:6379> SADD user:1002:skills "JavaScript" "Python" "React"
(integer) 3
```

READ Operations

A. String Operations

Enter the following command to get a single value

GET user:1001:name

Enter the following command to get multiple values at once

MGET user:1001:name user:1001:email user:1001:age

Enter the following command to check if key exists

EXISTS user:1001:name

Enter the following command to get all keys matching a pattern (use carefully in production!)

*KEYS user:1001:**

Expected Output:

```
127.0.0.1:6379> GET user:1001:name
"Alice Johnson"
127.0.0.1:6379> MGET user:1001:name user:1001:email user:1001:age
1) "Alice Johnson"
2) "alice@email.com"
3) "28"
127.0.0.1:6379> EXISTS user:1001:name
(integer) 1
127.0.0.1:6379> KEYS user:1001:*
1) "user:1001:age"
2) "user:1001:skills"
3) "user:1001:country"
4) "user:1001:name"
5) "user:1001:created_at"
6) "user:1001:email"
7) "user:1001:login_history"
8) "user:1001:notifi"
127.0.0.1:6379> HGETALL user:1002
1) "name"
2) "Bob Mwangi"
3) "email"
4) "bob@email.com"
```

B. Hash Operations

Enter the following command to get all fields and values from a hash

***HGETALL** user:1002*

Enter the following command to get specific field from hash

***HGET** user:1002 name*

Enter the following command to get multiple fields from hash

***HMGET** user:1002 name email role*

Enter the following command to get all field names

***HKEYS** user:1002*

Enter the following command to get all values

***HVALS** user:1002*

Enter the following command to check if field exists

***HEXISTS** user:1002 age*

Expected Outputs:

```
NEHEMA@DESKTOP-P67G70S MINGW64 ~/MIT8107-Lab-on-NoSQL-kifaru-byters (main)
$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> HGETALL user:1002
1) "name"
2) "Bob Mwangi"
3) "email"
4) "bob@email.com"
5) "age"
6) "32"
7) "country"
8) "Kenya"
9) "role"
10) "developer"
11) "department"
12) "Engineering"
127.0.0.1:6379> HGET user:1002 name
"Bob Mwangi"
127.0.0.1:6379> HMGET user:1002 name email role
1) "Bob Mwangi"
2) "bob@email.com"
3) "developer"
127.0.0.1:6379> HKEYS user:1002
1) "name"
2) "email"
3) "age"
4) "country"
5) "role"
6) "department"
127.0.0.1:6379> HVALS user:1002
1) "Bob Mwangi"
2) "bob@email.com"
3) "32"
```

C. List Operations

Enter the following command to get all elements from a list

LRange *user:1001:login_history 0 -1*

Enter the following command to get list length

LLEN *user:1001:login_history*

Enter the following command to get element at specific index

LINDEX *user:1001:login_history 0*

D. Set Operations

Enter the following command to get all members of a set

SMembers *user:1001:skills*

Enter the following command to check if member exists in set

SISMEMBER *user:1001:skills "Python"*

Enter the following command to get number of members

SCARD *user:1001:skills*

Enter the following command to find intersection of sets (common skills)

SINTER *user:1001:skills user:1002:skills*

Expected Output:

```
(base) joseph-vunanga@codegeek:~/Projects/MIT8107-Lab-on-NoSQL-kifaru-byters$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> LRange user:1001:login_history 0 -1
1) "2025-11-28 09:00:00"
2) "2025-11-28 14:30:00"
127.0.0.1:6379> LLEN user:1001:login_history
(integer) 2
127.0.0.1:6379> LINDEX user:1001:login_history 0
"2025-11-28 09:00:00"
127.0.0.1:6379> SMEMBERS user:1001:skills
1) "Python"
2) "Java"
3) "Docker"
4) "Redis"
127.0.0.1:6379> SISMEMBER user:1001:skills "Python"
(integer) 1
127.0.0.1:6379> SCARD user:1001:skills
(integer) 4
127.0.0.1:6379> SINTER user:1001:skills user:1002:skills
1) "Python"
```

UPDATE Operations

A. String Operations

Enter the following command to update existing value

SET *user:1001:name "Alice Wanjiku"*

Enter the following command to increment numeric value

SET *user:1001:login_count 5*

INCR *user:1001:login_count*

INCRBY *user:1001:login_count 10*

Enter the following command to decrement numeric value

DECR *user:1001:login_count*

DECRBY *user:1001:login_count 2*

Enter the following command to append to string value

SET *user:1001:bio "Data Scientist"*

APPEND *user:1001:bio " | AI Enthusiast"*

GET *user:1001:bio*

Expected Output:

```
NEHEMA@DESKTOP-P67G70S MINGW64 ~/MIT8107-Lab-on-NoSQL-kifaru-byters (main)
$ docker exec -it redis-lab redis-cli
1) "Python"
2) "Java"
3) "Docker"
4) "Redis"
127.0.0.1:6379> SISMEMBER user:1001:skills "Python"
(integer) 1
127.0.0.1:6379> SCARD user:1001:skills
(integer) 4
127.0.0.1:6379> SINTER user:1001:skills user:1002:skills
1) "Python"
127.0.0.1:6379> SET user:1001:name "Alice Wanjiku"
OK
127.0.0.1:6379> SET user:1001:login_count 5
OK
127.0.0.1:6379> INCR user:1001:login_count
(integer) 6
127.0.0.1:6379> INCRBY user:1001:login_count 10
(integer) 16
127.0.0.1:6379> DECR user:1001:login_count
(integer) 15
127.0.0.1:6379> DECRBY user:1001:login_count 2
(integer) 13
127.0.0.1:6379> SET user:1001:bio "Data Scientist"
OK
127.0.0.1:6379> APPEND user:1001:bio " | AI Enthusiast"
(integer) 30
127.0.0.1:6379> GET user:1001:bio
"Data Scientist | AI Enthusiast"
127.0.0.1:6379> HSET user:1002 age 33
(integer) 0
```

B. Hash Operations

Enter the following command to update field in hash

***HSET** user:1002 age 33*

Enter the following command to increment numeric field

***HINCRBY** user:1002 age 1*

Enter the following command to update multiple fields

***HMSET** user:1002 role "senior_developer" department "Platform Engineering"*

C. List Operations

Enter the following command to update element at specific index

***LSET** user:1001:login_history 0 "2025-11-28 09:15:00 (Updated)"*

D. Set Operations

Enter the following command to add new members to set

***SADD** user:1001:skills "Kubernetes" "AWS"*

Enter the following command to remove and add in one operation (not atomic, but sequential)

SREM user:1001:skills "Java"

SADD user:1001:skills "Golang"

DELETE Operations

A. Delete Keys

Enter the following command to delete a single key

DEL user:1001:created_at

Enter the following command to delete multiple keys

DEL user:1001:email user:1001:age

Enter the following command to delete key with expiration check and set a key with TTL

SET temp:key "temporary data" EX 10

TTL temp:key # Check time to live

(Wait 10 seconds and it auto-deletes)

Expected Output:

```
(base) joseph-vunanga@codegeek:~/Projects/MIT8107-Lab-on-NoSQL-kifaru-byters$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> DEL user:1001:created_at
(integer) 1
127.0.0.1:6379> DEL user:1001:email user:1001:age
(integer) 2
127.0.0.1:6379> SET temp:key "temporary data" EX 10
OK
127.0.0.1:6379> TTL temp:key
(integer) -2
```

B. Hash Operations

Enter the following command to delete specific fields from hash

HDEL user:1002 department

Enter the following command to delete entire hash

DEL user:1002

C. List Operations

Enter the following command to remove elements by value

LREM user:1001:notifications 1 "Welcome to the platform!"

Enter the following command to delete entire list

DEL user:1001:login_history

D. Set Operations

Enter the following command to remove member from set

SREM user:1001:skills "Docker"

Enter the following command to remove random member

SPOP user:1001:skills

Enter the following command to delete entire set

DEL user:1001:skills

4. Applied Scenario: Real-Time Session Management System

Problem Statement:

You're building a high-traffic web application (e.g., an e-learning platform) that needs to: Manage user sessions across multiple servers, Track active users in real-time, Implement rate limiting for API endpoints, Store temporary shopping cart data, Maintain real-time leaderboards for gamification

Traditional relational databases struggle with:

- ☐ High write throughput for session updates
- ☐ Sub-millisecond read latency requirements
- ☐ Automatic session expiration
- ☐ Horizontal scaling

Solution: Redis Key-Value Store

Redis excels at this scenario because:

- ☐ In-memory storage provides microsecond latency
- ☐ Built-in TTL (Time To Live) for automatic session expiration
- ☐ Atomic operations prevent race conditions
- ☐ Horizontal scalability through Redis Cluster
- ☐ Support for complex data structures (hashes, sets, sorted sets)

Implementation

Let's build a complete session management system step by step.

Step 1: User Authentication & Session Creation

When a user logs in, we create a session with a session id and store the session as a hash with 30-minute expiration. To avoid arguments errors, execute as one line command in redis cli:

```
HSET sess:550e8400-e29b-41d4-a716-446655440000 user_id "user:1001"  
username "alice@email.com" role "student" login_time "2025-11-28T10:30:00Z"  
ip_address "102.133.45.67" device "Chrome/MacOS"
```

Enter the following command to set session expiration (1800 secs = 30 min)

```
EXPIRE sess:550e8400-e29b-41d4-a716-446655440000 1800
```

Enter the following to add user to active users set (real-time tracking)

```
SADD active:users:2025-11-28 "user:1001"
```

Enter the following commands to track login event

```
RPUSH user:1001:login_events
```

```
"2025-11-28T10:30:00Z|102.133.45.67|Chrome/MacOS"
```

```
LTRIM user:1001:login_events -10 -1 # Keep last 10 login events
```

Enter the following command to increment daily login counter

```
INCR stats:logins:2025-11-28
```

Expected Output:

```
(base) joseph-vunanga@codegeek:~/Projects/MIT8107-Lab-on-NoSQL-kifaru-byters$ docker exec -it redis-lab redis-cli  
127.0.0.1:6379> HSET sess:550e8400-e29b-41d4-a716-446655440000 user_id "user:1001" username "alice@email.com" role "student" login_time  
e "2025-11-28T10:30:00Z" ip_address "102.133.45.67" device "Chrome/MacOS"  
(integer) 6  
127.0.0.1:6379> EXPIRE sess:550e8400-e29b-41d4-a716-446655440000 1800  
(integer) 1  
127.0.0.1:6379> SADD active:users:2025-11-28 "user:1001"  
(integer) 1  
127.0.0.1:6379> RPUSH user:1001:login_events "2025-11-28T10:30:00Z|102.133.45.67|Chrome/MacOS"  
(integer) 1  
127.0.0.1:6379> LTRIM user:1001:login_events -10 -1 # Keep last 10 login events  
(error) ERR wrong number of arguments for 'ltrim' command  
127.0.0.1:6379> LTRIM user:1001:login_events -10 -1  
OK  
127.0.0.1:6379> INCR stats:logins:2025-11-28  
(integer) 1
```

Step 2: Session Validation (Read)

In the real application, when user make a request, the session is validated by:

```
GET sess:550e8400-e29b-41d4-a716-446655440000 (do not run)
```

Since the session already exists, retrieve user details by running the following::

```
HGETALL sess:550e8400-e29b-41d4-a716-446655440000
```

Enter the following command to check remaining session time

```
TTL sess:550e8400-e29b-41d4-a716-446655440000
```

Enter the following command to get user's role for authorization

```
HGET sess:550e8400-e29b-41d4-a716-446655440000 role
```

Expected Output:

```
127.0.0.1:6379> HGETALL sess:550e8400-e29b-41d4-a716-446655440000
1) "user_id"
2) "user:1001"
3) "username"
4) "alice@email.com"
5) "role"
6) "student"
7) "login_time"
8) "2025-11-28T10:30:00Z"
9) "ip_address"
10) "102.133.45.67"
11) "device"
12) "Chrome/MacOS"
127.0.0.1:6379> TTL sess:550e8400-e29b-41d4-a716-446655440000
(integer) 1534
127.0.0.1:6379> HGET sess:550e8400-e29b-41d4-a716-446655440000 role
"student"
```

Step 3: Session Activity Update

After refreshing the session (EXPIRE sess:session_id duration), enter the following command to update last activity timestamp

```
HSET sess:550e8400-e29b-41d4-a716-446655440000 last_activity
"2025-11-28T10:45:00Z"
```

Enter the following command to track page views

```
INCR user:1001:page_views:2025-11-28
```

Step 4: Shopping Cart Management

Enter the following command to add items to shopping cart (using hash)

```
HSET cart:user:1001 item:course101 "1/299.99/Introduction to Python"
item:course205 "1/399.99/Advanced Data Structures" item:book42
"2/29.99/Redis Essentials"
```

Enter the following command to set cart expiration (24 hours)

```
EXPIRE cart:user:1001 86400
```

Enter the following command to get cart contents

```
HGETALL cart:user:1001
```

Enter the following command to update item quantity

```
HSET cart:user:1001 item:book42 "3/29.99/Redis Essentials"
```

Enter the following command to remove item from cart

```
HDEL cart:user:1001 item:course101
```

Enter the following command to calculate cart total (application logic)

```
HVALS cart:user:1001
```

Expected Output:

```
(base) joseph-vunanga@codegeek:~/Projects/MIT8107-Lab-on-NoSQL-kifaru-byters$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> HSET sess:550e8400-e29b-41d4-a716-446655440000 last_activity "2025-11-28T10:45:00Z"
(integer) 1
127.0.0.1:6379> INCR user:1001:page_views:2025-11-28
(integer) 1
127.0.0.1:6379> HSET cart:user:1001 item:course101 "1|299.99|Introduction to Python" item:course205 "1|399.99|Advanced Data Structures" item:book42 "2|29.99|Redis Essentials"
Invalid argument(s)
127.0.0.1:6379> HSET cart:user:1001 item:course101 "1|299.99|Introduction to Python" item:course205 "1|399.99|Advanced Data Structures" item:book42 "2|29.99|Redis Essentials"
(integer) 3
127.0.0.1:6379> EXPIRE cart:user:1001 86400
(integer) 1
127.0.0.1:6379> HGETALL cart:user:1001
1) "item:course101"
2) "1|299.99|Introduction to Python"
3) "item:course205"
4) "1|399.99|Advanced Data Structures"
5) "item:book42"
6) "2|29.99|Redis Essentials"
127.0.0.1:6379> HSET cart:user:1001 item:book42 "3|29.99|Redis Essentials"
(integer) 0
127.0.0.1:6379> HDEL cart:user:1001 item:course101
(integer) 1
127.0.0.1:6379> HVALS cart:user:1001
1) "1|399.99|Advanced Data Structures"
2) "3|29.99|Redis Essentials"
```

Step 5: Rate Limiting (API Throttling)

Let's implement rate limiting: 100 requests per minute per user. We will use the following key format: *ratelimit:user_id:minute*. Enter the following command to record request:

INCR ratelimit:user:1001:2025-11-28:10:45

EXPIRE ratelimit:user:1001:2025-11-28:10:45 60

Enter the following command to check if limit exceeded

GET ratelimit:user:1001:2025-11-28:10:45

Enter the following command to simulate multiple requests

INCR ratelimit:user:1001:2025-11-28:10:45

INCR ratelimit:user:1001:2025-11-28:10:45

GET ratelimit:user:1001:2025-11-28:10:45

Expected Output:

```
127.0.0.1:6379> INCR ratelimit:user:1001:2025-11-28:10:45
(integer) 1
127.0.0.1:6379> EXPIRE ratelimit:user:1001:2025-11-28:10:45 60
(integer) 1
127.0.0.1:6379> GET ratelimit:user:1001:2025-11-28:10:45
"1"
127.0.0.1:6379> INCR ratelimit:user:1001:2025-11-28:10:45
(integer) 2
127.0.0.1:6379> INCR ratelimit:user:1001:2025-11-28:10:45
(integer) 3
127.0.0.1:6379> INCR ratelimit:user:1001:2025-11-28:10:45
(integer) 4
127.0.0.1:6379> INCR ratelimit:user:1001:2025-11-28:10:45
(integer) 5
127.0.0.1:6379> GET ratelimit:user:1001:2025-11-28:10:45
"5"
```

Step 6: Real-Time Leaderboard (Sorted Sets)

We will now track student points using sorted sets.

The Score represents total points:

```
ZADD leaderboard:november:2025 850 "user:1001" 920 "user:1002" 780  
"user:1003" 1050 "user:1004" 890 "user:1005"
```

Enter the following command to get top 3 students (highest scores)

```
ZREVRANGE leaderboard:november:2025 0 2 WITHSCORES
```

Enter the following command to get student's rank (0-indexed, from highest)

```
ZREVRANK leaderboard:november:2025 "user:1001"
```

Enter the following command to get student's score

```
ZSCORE leaderboard:november:2025 "user:1001"
```

Enter the following to increment student's score (earned 50 points)

```
ZINCRBY leaderboard:november:2025 50 "user:1001"
```

Enter the following command to get students within score range

```
ZRANGEBYSCORE leaderboard:november:2025 800 900 WITHSCORES
```

Enter the following command to get total number of students

```
ZCARD leaderboard:november:2025
```

Expected Output:

```
127.0.0.1:6379> ZADD leaderboard:november:2025 850 "user:1001" 920 "user:1002" 780 "user:1003" 1050 "user:1004" 890 "user:1005"  
(integer) 5  
127.0.0.1:6379> ZREVRANGE leaderboard:november:2025 0 2 WITHSCORES  
1) "user:1004"  
2) "1050"  
3) "user:1002"  
4) "920"  
5) "user:1005"  
6) "890"  
127.0.0.1:6379> ZREVRANK leaderboard:november:2025 "user:1001"  
(integer) 3  
127.0.0.1:6379> ZSCORE leaderboard:november:2025 "user:1001"  
"850"  
127.0.0.1:6379> ZINCRBY leaderboard:november:2025 50 "user:1001"  
"900"  
127.0.0.1:6379> ZRANGEBYSCORE leaderboard:november:2025 800 900 WITHSCORES  
1) "user:1005"  
2) "890"  
3) "user:1001"  
4) "900"  
127.0.0.1:6379> ZCARD leaderboard:november:2025  
(integer) 5
```

Step 7: Session Logout (Delete)

When the user logs out - delete session

```
DEL sess:550e8400-e29b-41d4-a716-446655440000
```

Enter the following command to remove from active users

```
SREM active:users:2025-11-28 "user:1001"
```

Optional: Enter the following to clear shopping cart if not persistent

```
DEL cart:user:1001
```


Enter the following commands to logout

RPUSH user:1001:login_events "2025-11-28T11:15:00Z/LOGOUT"

LTRIM user:1001:login_events -10 -1

Step 8: Analytics Queries

Get count of active users today

SCARD active:users:2025-11-28

Get total logins today

GET stats:logins:2025-11-28

Get user's page views today

GET user:1001:page_views:2025-11-28

Get recent login events for user

LRANGE user:1001:login_events 0 -1

Get all active sessions (pattern matching - use carefully!)

*KEYS sess:**

Scenario Performance Benefits:

Comparison: Redis vs Traditional RDBMS

Operation	Redis(Key-Value)	PostgreSQL(RDBMS)
Session Write	< 1ms	5-20ms
Session Read	< 1ms	3-15ms
Leaderboard Query	< 1ms	50-200ms
Rate Limit Check	< 1ms	10-30ms
Auto-Expiration	Native (TTL)	Requires cleanup jobs
Concurrent Users	Millions	Thousands

Real-World Impact:

- ☐ Latency: 10-20x faster response times
- ☐ Throughput: Handle 100,000+ req/sec per instance
- ☐ Simplicity: No complex queries or indexes needed
- ☐ Cost: Lower server requirements due to efficiency

5. Advanced Operations

Transactions (MULTI/EXEC)

We can achieve atomic operations -all or nothing- using transactions:

MULTI

```
SET account:1001:balance 1000  
DECRBY account:1001:balance 100  
INCRBY account:1002:balance 100
```

EXEC

Pipelining (Batch Operations)

We can send multiple commands without waiting for responses (Reduce network round trips). In redis-cli, use the *--pipe* flag. In application code, use the pipeline feature. Eg: Enter the following command to create 5 users

MULTI

```
SET user:2001 "User 2001"  
SET user:2002 "User 2002"  
SET user:2003 "User 2003"  
SET user:2003 "User 2004"  
SET user:2003 "User 2005"
```

EXEC

Expected Output

```
127.0.0.1:6379> MULTI  
OK  
127.0.0.1:6379(TX)> SET account:1001:balance 1000  
QUEUED  
127.0.0.1:6379(TX)> DECRBY account:1001:balance 100  
QUEUED  
127.0.0.1:6379(TX)> INCRBY account:1002:balance 100  
QUEUED  
127.0.0.1:6379(TX)> EXEC  
1) OK  
2) (integer) 900  
3) (integer) 100  
127.0.0.1:6379> MULTI  
OK  
127.0.0.1:6379(TX)> SET user:2001 "User 2001"  
QUEUED  
127.0.0.1:6379(TX)> SET user:2002 "User 2002"  
QUEUED  
127.0.0.1:6379(TX)> SET user:2003 "User 2003"  
QUEUED  
127.0.0.1:6379(TX)> SET user:2003 "User 2004"  
QUEUED  
127.0.0.1:6379(TX)> SET user:2003 "User 2005"  
Invalid argument(s)  
127.0.0.1:6379(TX)> EXEC  
1) OK  
2) OK  
3) OK  
4) OK
```

Pub/Sub (Real-Time Notifications)

Open 2 command prompts to simulate the following scenario. In Terminal 2, you will subscribe to notifications:

SUBSCRIBE *user:1001:notifications*

In Terminal 1, you will publish notification

PUBLISH *user:1001:notifications "Your course has been approved!"*

Expected Output:

```
sharonleah@Sharons-MacBook-Air MIT8107-Lab-on-NoSQL-kifaru-byters % docker exec -it redis-lab redis-cli
127.0.0.1:6379> PUBLISH user:1001:notifications "Your course has been approved!"
(integer) 0
127.0.0.1:6379> PUBLISH user:1001:notifications "Your course has been approved!"
(integer) 1
```

```
127.0.0.1:6379> SUBSCRIBE user:1001:notifications
1) "subscribe"
2) "user:1001:notifications"
3) (integer) 1
1) "message"
2) "user:1001:notifications"
3) "Your course has been approved!"
```

Data Persistence Check

Enter the following command to save current state to disk

SAVE

Enter the following command to get last save time

LASTSAVE

Enter the following command to enable background save

BGSAVE

In order to check persistence configuration, use the following commands:

CONFIG GET *save*

CONFIG GET *appendonly*

Performance Monitoring

Enter the following command to get server information

INFO

To Monitor commands in real-time, run the following in the 2nd terminal. Any prompts you run in the first terminal would be displayed here as logs

MONITOR

Enter the following command to get statistics, check the 2nd terminal for logs

INFO stats

Enter the following command to get memory usage, check 2nd terminal for any logs

INFO memory

MEMORY USAGE *user:1001:login_events*

6. Group Contribution Summary

Optional: Remove Test Data

Enter the following command to delete all keys in the current database (BE CAREFUL!)

FLUSHDB

Or enter the following command to delete specific patterns

DEL user:1001:name user:1001:bio cart:user:1001

Enter the following command to exit Redis CLI

EXIT

Stop Docker Container

Enter the following command to stop the container

docker stop redis-lab

Key Takeaways

1. Use Meaningful Key Names: Follow naming conventions (*object:id:field*)
2. Set Appropriate TTLs: Prevent memory bloat with expiration
3. Monitor Memory Usage: Use *INFO memory* regularly
4. Use Hashes for Objects: More memory efficient than multiple string keys
5. Implement Application-Level Validation: Redis doesn't enforce schemas

Team Members and Responsibilities:

Student ID	Member Name	Contribution
112721	Kathembo Tsongo	Setup Instructions, GitHub+Docker Config
226022	Kavira Neema Nancy	CRUD operations
225637	Olang Sharon Leah	Advanced Operations
136371	Joseph Vunanga	Applied Scenario Implementation

Appendix: Troubleshooting

Common Issues:

1. Cannot connect to Redis:

Enter the following command to check if container is running

```
docker ps
```

Enter the following command to check logs

```
docker logs redis-lab
```

Enter the following command to restart container

```
docker restart redis-lab
```

2. Connection refused error:

Enter the following command to verify port mapping

```
docker port redis-lab
```

Enter the following command to check firewall settings on **Ubuntu**

```
sudo ufw status
```

3. Out of memory errors:

Enter the following command to check memory usage

```
INFO memory
```

Enter the following command to clear unnecessary data

```
FLUSHDB
```

Enter the following command to configure max memory

```
CONFIG SET maxmemory 256mb
```

```
CONFIG SET maxmemory-policy allkeys-lru
```

References

Further Learning Materials:

1. Official Documentation: <https://redis.io/docs/>
2. Redis University: Free courses at <https://university.redis.com/>
3. Redis Commands Reference: <https://redis.io/commands/>
4. Try Redis Online: <https://try.redis.io/>